Test Summary

No. of Sections: 1
Total Questions: 15
Total Duration: 40 mins

Section 1- Code Based Test

Section Summary

No. of Questions: 8
Duration: 40 mins

Fill-ups (4 Questions)

Q1. What should replace the blank for type promotion?

```
public class TypePromotion {
   public static void main(String[] args) {
       byte b = 10;
       int i = 20;
       ______ result = b + i;
       System.out.println(result);
   }
}
```

a) `byte`

b) `short`

c) `int`

d) `long`

Q2. What operator should replace the blank for less than?

```
public class LessThan {
   public static void main(String[] args) {
       int a = 5, b = 10;
       boolean less = a ______ b;
       System.out.println(less);
   }
}
```

a) <

b) >

c) <=

d) >=

Q3. What should replace the blank to print numbers 1 to 5 using while loop?

```
public class WhileLoop {
   public static void main(String[] args) {
       int i = 1;
       while(i ______ 5) {
           System.out.println(i);
           i++;
       }
   }
}
```

a) >

b) <

c) <=

d) ==

Q4. What should replace the blank for constructor overloading?

```
public class Circle {
   double radius;

   public Circle() {
       radius = 1.0;
   }

   public ______(double r) {
       radius = r;
   }

   public static void main(String[] args) {
       Circle c1 = new Circle();
       Circle c2 = new Circle(5.0);
   }
}
```

a) `Circle`

b) `Constructor`

c) `initCircle`

d) `CircleConstructor`

Q5. What should replace the blank to avoid infinite loop in while?

```
public class WhileIncrement {
   public static void main(String[] args) {
       int i = 0;
       while(i < 5) {
           System.out.println(i);
           i ______;
       }
   }
}
```

a) `+`

b) `++`

c) `+=`

d) `= j + 1`

Issue Code (4 Questions)

Q6) Identify the correct code after fixing the error to get the expected output: "Initial: 0"

```
public class DefaultValues {
  static int count;
  public static void main(String[] args) {
    int value; // Issue
    System.out.println("Initial: " + value);
  }
}
```

a) int value = 0;

b) static int value;

c) Integer value = 0;

d) int value = count;

Q7) Identify the correct code after fixing the error to get the expected output: "Number is 5"

```
public class IfAssignment {
   public static void main(String[] args) {
      int number = 5;
      if (number = 5) { // Issue
         System.out.println("Number is 5");
      }
   }
}
```

```
a) if (number == 5)
b) if (number + 5)
c) if (!number)
d) if (number.equals(5))
```

Q8) The code throws a NullPointerException at runtime. Which correction fixes the error in line 8?

```
public class ObjectArray {
   public static void main(String[] args) {
      String[] names = new String[3];
      names[0] = "Alice";
      names[1] = "Bob";

      for (int i = 0; i < names.length; i++) {
         System.out.println(names[i].toUpperCase()); // Issue
      }
   }
}
```

```
a) if (names[i] != null) System.out.println(names[i].toUpperCase());
b) System.out.println(names[i].toString().toUpperCase());
c) System.out.println(names[i].toLowerCase().toUpperCase());
d) System.out.println(names[j].toUpperCase());
```

Q9) The code doesn't remove spaces from the original string. Which correction fixes the error in line 4?

```
public class StringTrim {
   public static void main(String[] args) {
      String str = "  Hello World  ";
      str.trim(); //Issue
      System.out.println("[" + str + "]");
```

```
        String trimmed = str.trim();
        System.out.println("[" + trimmed + "]");
    }
}
```

a) str = str.trim();
b) str.trim().toString();
c) String newStr = str.trim();
d) str = str;

Q10) The array sorting logic is inefficient and incorrect. Which correction fixes the error in line 6?

```
import java.util.Arrays;

public class ArraySort {
    public static void main(String[] args) {
        int[] nums = {5, 2, 8, 1, 9};

        for (int i = 0; i < nums.length; i++) {
            for (int j = 0; j < nums.length; j++) { // Issue
                if (nums[i] > nums[j]) {
                    int temp = nums[i];
                    nums[i] = nums[j];
                    nums[j] = temp;
                }
            }
        }
        System.out.println("Sorted: " + Arrays.toString(nums));
    }
}
```

Options:
a) for (int j = i + 1; j < nums.length; j++) {
b) for (int j = 0; j > nums.length - 1; j++) {
c) for (int j = i; j < nums.length; j++) {
d) for (int j = 0; j < nums.length; j--) {

Q11) The code incorrectly splits a string containing dots. Which correction fixes the error in line 11?

```
public class StringSplit {
  public static void main(String[] args) {
    String data = "apple,banana,orange";
    String[] fruits = data.split(",");

    for (String fruit : fruits) {
      System.out.println(fruit);
    }

    String path = "a.b.c";
    String[] parts = path.split("."); // Issue
    System.out.println(parts.length);
  }
}
```

Options:

a) String[] parts = path.split("\\.");

b) String[] parts = path.split("/.");

c) String[] parts = path.split("..");

d) String[] parts = path.split("*");

Multiple Correct (3 Questions)

Q12) Which snippets correctly demonstrate encapsulation?

a) public class BankAccount { private double balance;

```
public double getBalance() {
  return balance;
}


public void deposit(double amount) {
  if (amount > 0) {
    balance += amount;
  }
}


public boolean withdraw(double amount) {
  if (amount > 0 && amount <= balance) {
    balance -= amount;
    return true;
  }
  return false;
}


}
```

b) public class Student {

```
  public String name;  // public field - poor encapsulation

  public int age;
}
```

c) public class Rectangle { private double width; private double height;

```
public Rectangle(double width, double height) {

  setWidth(width);

  setHeight(height);

}


public void setWidth(double width) {

  if (width > 0) this.width = width;

}


public void setHeight(double height) {

  if (height > 0) this.height = height;

}


}
```

d) public class Person {

```
  private String ssn;  // Social Security Number


  public String getSSN() {  // exposes sensitive data

     return ssn;

  }

}
```

Options:

1) a,c

2) a,b,d

3) b,c,d

4) a,b,c

Q13) Which snippets correctly demonstrate nested loop patterns?

a) for (int i = 1; i <= 5; i++) { for (int j = 1; j <= i; j++) { System.out.print("*"); } System.out.println(); }

b)
```
for (int i = 5; i >= 1; i--) {
  for (int j = 1; j <= i; j++) {
     System.out.print(j + " ");
  }
  System.out.println();
}
```

c) int rows = 3, cols = 4; for (int i = 0; i < rows; i++) { for (int j = 0; j < cols; j++) { System.out.print("(" + i + "," + j + ") "); } System.out.println(); }

d)

```
for (int i = 0, j = 0; i < 3 && j < 3; i++, j++) {
   System.out.println(i + " " + j);
}
```

Options:

1) a,b,c

2) a,b,d

3) b,c,d

4)a,b,c,d

Q14) Which snippets correctly demonstrate ternary operator?

a) int a = 10, b = 20;
int max = (a > b) ? a : b;
String result = (a > b) ? "a larger" : "b larger or equal";

b) int score = 85;
String grade = (score >= 90) ? "A" :
(score >= 80) ? "B" :
(score >= 70) ? "C" :
(score >= 60) ? "D" : "F";

c) String text = null;
String output = (text != null) ? text.toUpperCase() : "DEFAULT";

d) int x = 5;
int y = (x > 0) ? "positive" : "negative"; // type mismatch

Options:

1) a,b,c

2) a,b,d

3) b,c,d

4) a,b,c,d

Q15) Which snippets correctly demonstrate overflow/underflow?

a) int max = Integer.MAX_VALUE;
int overflow = max + 1; // becomes Integer.MIN_VALUE
System.out.println(overflow);

B) int min = Integer.MIN_VALUE;
int underflow = min - 1; // becomes Integer.MAX_VALUE
System.out.println(underflow);

c) byte b = 127;
b = b + 1;
b = (byte)(b + 1);

d) double d1 = 1.0e308 * 2.0;
double d2 = 1.0e-324 / 2.0;

Options:

1) a,b,d

2) a,b,c

3) b,c,d

4) a, b, c, d